SR-MPLS Training Lab

M7 End-to-End QoS Marking

Estimated time: 60 min

Objective

Follow one packet from a setsockopt() call in application source code, through the IPv4 TOS byte, across the customer LAN, into a provider edge that decides whether to believe the marking, down into three bits of MPLS EXP across the core, and back out the far side into a DSCP the receiving application reads off its own socket. By the end you can explain why sixty-four DSCP values cannot survive a network that carries eight, and why an application does not get to decide its own priority.

Before you start: M5 complete, so the class-of-service framework and the EXP rewrite are already understood, and the full SR-MPLS stack running. Both Linux endpoints reachable with the qos-e2e application tree deployed to /home/rocky/qos-e2e (see qos-e2e/deploy_clients.py).

Concept primer

An application marks itself with one line: setsockopt(IPPROTO_IP, IP_TOS, dscp << 2). That value lands in the IPv4 header's TOS byte, nowhere else. It does not touch the transport layer; ports and protocol are separate fields the network can match on independently. The shift by two is because RFC 2474 redefined the byte so DSCP owns the top six bits and ECN owns the bottom two. DSCP is not married to TOS, it is part of it.

Six bits hold sixty-four values. The MPLS EXP field, also called Traffic Class, holds three bits and therefore eight. When the ingress PE imposes a label stack it must collapse whatever DSCP arrived into one of a handful of forwarding classes, and only that class crosses the core. Every transit router in this lab reads the label and never looks at the IP header at all. On the far side the egress PE rebuilds a DSCP from the forwarding class, which means it cannot recover which of several DSCPs originally mapped into that class. The round trip is lossy, and this module makes you watch it happen.

There are two kinds of classifier and the difference is the whole trust conversation. A behaviour-aggregate (BA) classifier maps the DSCP that arrives to

a forwarding class: it believes the customer completely. A multifield (MF) classifier is a firewall filter that assigns a forwarding class based on the flow's identity, five-tuple and all, regardless of what the packet claimed. Applied in the input direction, its forwarding-class action overrides the BA result. That override is where 'my application marks itself important' stops being true.

One more thing worth knowing before you touch a router: this lab's PFE is software-simulated (vJunos in lite-mode), and it turns out to genuinely differ from hardware in two ways that matter for this module. It does not expose per-queue counters, so you cannot watch a Transmitted count climb to confirm placement, and its egress shaper does not actually enforce a rate under load even though the control plane accepts and recomputes against it. Both limitations are covered where they come up below, with the true commands to use instead.

Build steps

1. **Read the marking line in the application source. Every sender uses the same helper, so this one line is the entire claim an application can make about its own importance.**

```
client1$ grep -A8 'def set_dscp' ~/qos-e2e/isr_apps/common.py
client1$ grep -n 'set_dscp' ~/qos-e2e/isr_apps/*.py
```

Note: Four different applications, one identical line. Show this to a software audience before anything else: it is the only part of the chain they control, and it is smaller than they expect.

2. **Prove the TOS byte actually reaches the wire.**

```
client1$ sudo python3 ~/qos-e2e/isr_apps/wire_peek.py --iface eth0 --count 5 --port 9100 &
client1$ python3 ~/qos-e2e/isr_apps/c2_messaging.py --duration 5
```

Note: wire_peek prints 'CS6 (48)' where tcpdump prints 'tos 0xc0'. Same information, readable by an audience that does not think in hex. This is the last point in the chain where the application's wish and reality are guaranteed to agree.

3. **Push stage 7 and read the two classifiers side by side.**

```
python push_lab_configs.py stage7-qos-e2e
show class-of-service classifier name DSCP-TRUSTED
show configuration firewall family inet filter UNTRUSTED-REMARK
show class-of-service interface ge-0/0/3
```

Note: DSCP-TRUSTED maps six customer DSCPs onto four forwarding classes. UNTRUSTED-REMARK matches TCP port 9300 and forces best-effort with 'then dscp

cs1', overriding whatever the BA classifier decided. Both are bound to ge-0/0/3.100. One trusts, one does not.

4. **CRITICAL: restart routing on every node in the topology, not only the ones this stage touched.**

```
## On EVERY router: PE1, PE2, P1, P2, P3, P4, C1, C2
restart routing
```

Note: This is the single most expensive lesson this lab build produced. After any stage push that changes segment-routing configuration, the SRGB does not take effect until routing is restarted, and it must be restarted on every node, not just the ones you edited. Skip one node and the lab looks completely healthy: IS-IS adjacencies Up, BGP established, no alarms, while forwarding is silently dead and every VPN route sits with an unresolvable next hop. This cost this build several hours to diagnose. If the next step does not work, check this before you doubt the classifier configuration.

5. **Run the console on Client2, then all four applications on Client1 with the imagery push in greedy mode.**

```
client2$ python3 ~/qos-e2e/console/render_tui.py
client1$ cd ~/qos-e2e/isr_apps && python3 c2_messaging.py --duration 45 & python3 track_feed.py --duration 45 & python3 fmv_stream.py --duration 45 & python3 imagery_push.py --greedy --duration 45 & wait
```

Note: The console reads each arriving datagram's TOS byte back off its own socket with IP_RECVTOS and compares it against what the sender recorded in the payload header. Watch the VERDICT column. Two rows are the point of the whole module. That IP_RECVTOS path only applies to the three UDP rows; imagery_push is TCP, and this lab's kernel never delivers IP_RECVTOS ancillary data for a stream socket, so its arrived DSCP is read instead with getsockopt(IPPROTO_IP, IP_PKTOPTIONS, ...), the console's TCP-specific fallback.

6. **Capture a core link and read the EXP bits for all four streams.**

```
## Right-click the PE1 to P1 link in GNS3, choose 'Start capture'.
## Re-run the senders, then open the capture in Wireshark.
## Expand MultiProtocol Label Switching Header for each stream.
```

Note: The FMV stream and the track feed are indistinguishable here. They chose different DSCPs and both carry EXP 100, because three bits cannot hold sixty-four values. This is the collapse on the wire, and it is the check the router CLI alone cannot give you. Do not expect 'monitor traffic interface ge-0/0/1' on the router itself

to show any of this: monitor traffic is a routing-engine-side capture and only ever sees control-plane packets addressed to the RE, never transit customer traffic. Four attempts across both core uplinks with tens of thousands of customer packets in flight captured none of them. The GNS3 GUI capture above is the only way to see EXP for customer data traffic in this lab, so do not lose an hour rediscovering that on your own.

7. **Toggle the egress rewrite on PE2 to move between the RFC 3270 tunnel models and watch the delivered DSCP change.**

```
## On PE2:
delete class-of-service interfaces ge-0/0/3 unit 100 rewrite-rules dscp EGRESS-DSCP
commit
## Re-run track_feed and re-read the console, then restore:
rollback 1
commit
```

Note: With the rewrite applied you are in the uniform model: the delivered DSCP is rebuilt from the forwarding class, so AF31 comes out as AF41. With it removed you are closer to the pipe model: the customer's original DSCP survives untouched end to end. Same network, different contract with the customer, one config line.

8. **Push a bottleneck onto a core link and find out whether marking buys any protection under real congestion.**

```
## GNS3 2.2.59 has no bandwidth filter; the only link filters available are frequency_drop,
packet_loss, delay (latency and jitter), corrupt, and bpf.
## Right-click the PE1 to P1 link, Packet filters:
##   Packet loss 5 %, Latency 80 ms, Jitter 20 ms, then Apply
## Watch the console loss and jitter columns for 30 seconds.
## Clear the filters and confirm recovery.
```

Note: GNS3 link filters are DSCP-blind: they degrade every packet on the link equally, so they can prove the network got worse but never that premium traffic was protected while bulk traffic was not. The real answer to that question was measured separately with class-of-service shaping (qos-e2e/spike_scheduling.md): 'shaping-rate 5m' committed cleanly on PE1's core interface, Junos even recomputed the scheduler's bandwidth math against it, and two competing flows, one EF-marked and one unmarked, were then offered at 20 Mbps each, 40 Mbps combined into that 5 Mbps ceiling. Both flows sustained their full offered rate through the shaper for the whole 30-second test with zero percent loss. The control plane confirmed a policy the forwarding plane never applied. This lab does not demonstrate protection under congestion, and it should not be shown as if it does. That gap is exactly why you test

forwarding behaviour rather than trust 'show' output, and it is a more honest lesson for a software audience than a scripted demo that quietly cannot fail.

Verification

The console ledger on Client2

```
STREAM        ASKED       ARRIVED      PACKETS    LOSS    JITTER  VERDICT
-------------------------------------------------------------------------------
c2_messaging   CS6 (48)     CS6 (48)         90    0.0%     4.8ms  honored
fmv_stream     AF41 (34)    AF41 (34)      4493    0.2%     5.8ms  honored
imagery_push   EF (46)      CS1 (8)       60313     n/a      n/a  RE-MARKED AT GATEWAY
track_feed     AF31 (26)    AF41 (34)       899    0.1%     4.7ms  CLASS COLLAPSED
```

What to look for: This is the real console output this lab produced (re-measured 2026-08-06 after commit 52daca5's TCP reassembly fix, with the column header exactly as render_tui.render() emits it: ARRIVED, not DELIVERED), not an approximation. imagery_push's PACKETS column (60313) matches the sender's own reported chunk count exactly on this run; that agreement is the evidence that TCP reassembly is working, not merely plausible. c2_messaging and fmv_stream were both delivered exactly what they asked for: honored. track_feed asked for AF31 and was delivered AF41: CLASS COLLAPSED, and nothing is broken. AF31 and AF41 both collapse into assured-forwarding at the ingress PE, only the class crosses the core, and the egress rewrite rebuilds a DSCP from the class alone with no way to know which of the two original values went in. imagery_push claimed EF, a real class in this network with genuine strict-high scheduling, and was delivered CS1 anyway: RE-MARKED AT GATEWAY, because the untrusted-remark filter classifies on flow identity rather than on the claim. Its loss and jitter columns read n/a deliberately: imagery_push runs over TCP, and TCP does not lose data, it retransmits, so a loss percentage there would describe retransmissions rather than delivery and would be meaningless next to the UDP-style figures on the other three rows. That distinction is worth making to a software audience on its own.

show class-of-service interface ge-0/0/3 (on PE1)

```
Logical interface: ge-0/0/3.100
  Classifier   DSCP-TRUSTED   dscp
```

What to look for: The BA classifier is bound to the customer-facing unit. If this is missing, every customer DSCP falls to best-effort and all four streams receive identical treatment no matter what they marked. This is also the check to lean on instead of queue counters: this platform's 'show interfaces queue' returns no per-queue data at all (see M5's verify section for the full explanation), so classifier and

rewrite-rule configuration plus a packet capture are the only way to confirm where a packet actually landed.

python qos_trace.py track_feed

```
1  application asks       Client1          AF31 (26)
4  gateway decides         PE1 ge-0/0/3.100   BA=on MF=on -> assured-forwarding
5  collapse into label     PE1 core egress    EXP 100
7  reverse translation      PE2 ge-0/0/3.100   rewrite=on -> DSCP 34
```

What to look for: The whole chain on one screen. Stage 4 shows whether both classifiers are in force, and stage 7 shows the delivered DSCP differing from the one the application asked for.

[screenshot 008: M7 verification: the console ledger showing collapse and re-mark]

Break / Fix exercise

Scenario: An audit reports that a bulk imagery transfer is receiving Expedited Forwarding treatment end to end, ahead of command and control traffic. Every IS-IS adjacency is Up, the L3VPN is healthy, the core is marking EXP faithfully, and the DSCP classifier on the ingress PE is present and correct. The application team confirms their transfer does set EF on its socket, and insists this is legitimate because the classifier accepts the value.

Your task: Restore correct treatment so the bulk transfer is carried as best effort regardless of what it marks, without changing the behaviour of any other stream and without asking the application team to change their code. Prove the fix from the receiving application's own output, not only from the router CLI.

Hint: A behaviour-aggregate classifier is not a security control. It maps whatever DSCP arrives to a forwarding class and believes the customer completely, which is exactly what it is designed to do. Something else has to disbelieve them. That something acts on the flow's identity rather than its marking, and it must be applied in the input direction on the customer-facing logical unit to take effect before classification. Compare 'show class-of-service interface ge-0/0/3' and 'show configuration interfaces ge-0/0/3 unit 100' on the box against the generated configs/stage7-qos-e2e/PE1.set and look for the object that exists in the file but not on the router. Queue counters will not flag this on their own, because this platform does not expose per-queue counters at all, and even where placement can be shown, placement follows classification, and the classification is doing exactly what it was told.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Trace one track_feed packet end to end. Name the DSCP it carries leaving the application, the forwarding class it is assigned at the ingress PE, the EXP value it carries on a core link, and the DSCP it is delivered with. Then explain why the first and the last of those differ, without using the word 'bug'.
- Explain the difference between a behaviour-aggregate classifier and a multifield classifier. Say which one enforces a trust boundary, why the other one cannot, and what would happen if you deployed only the first.
- A software team asks you to guarantee their application priority by having it mark itself EF. Give them a complete answer: what marking does and does not achieve, where the decision actually gets made, how many distinct treatments the core can express, and what they would need to agree with the network team for their marking to mean anything at all.